Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Designing the Server-Side REST API Flow and Initial API Contract for Your Own Project

Server-Side and REST API Concepts

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 7
Output	API flow diagram, request-lifecycle trace, initial API contract, endpoint and validation plan
Maximum Score	80 points

Student / Project Information

Block: 3A	Date: October 06, 2026
System Title: Humanitarian Aid Distribution Management System with Machine Learning-Based Household Vulnerability Classification	Instructor: Jean Y. Villasin, MIT
Members: Steven P. Payapaya Kyle M. Borja	

1. Activity Alignment and Purpose

Topic Learning Outcome: Explain the role of the REST API backend and design the request–response contract that links Flutter Web to the PostgreSQL data layer.

Course Outcome Alignment: Supports semester-project server-side planning by connecting the approved Flutter Web client design to an instructor-approved REST API backend and the PostgreSQL data layer developed in IT 108.

Syllabus Assessment: API Flow Diagram; Initial API Contract; online request–response tracing exercise.

Purpose of the Activity

Activity 6 established the Flutter Web UI flow and interface behavior. In Activity 7, your team will define what happens after the Flutter client sends a request: which endpoint receives it, how the server validates and authorizes it, which service/business rule executes, how PostgreSQL data is accessed, and what structured response returns to the client. The API flow diagram and initial API contract will become the technical bridge between your Flutter Web screens and the data layer implemented in IT 108.

Important distinction: This activity designs the server-side flow and API contract. It does not duplicate database modeling, normalization, constraints, indexes, transactions, stored procedures, or other database-design work completed in IT 108.

2. Intended Learning Objectives

- Explain the responsibility of the server-side application and REST API layer in the complete web-system architecture.
- Plan REST endpoints, routes, controllers/handlers, service-layer responsibilities, and conceptual data-access responsibilities for the team's actual system.
- Separate transport concerns, server-side validation, authorization, business logic, and data access into clear responsibilities.
- Define JSON request and response bodies, HTTP methods, status codes, required headers, and structured error responses.
- Trace one important request from the Flutter Web interface through the REST API to PostgreSQL and back to the user interface.
- Produce an initial API contract that is traceable to the approved requirements, use cases, UI flow, and PostgreSQL data domains.

3. Activity Specifications

Mode	Existing semester-project group; all members must participate and understand the API flow.
Suggested Time	One 2-hour laboratory session, with outside time allowed for refinement before submission.
Primary Output	One API Flow Diagram for the team's own system.
Supporting Output	Request-lifecycle trace, endpoint inventory, initial API contract, server-validation/error plan, and conceptual API-to-PostgreSQL mapping.
Recommended Tools	diagrams.net/Draw.io, Figma, Lucidchart, PowerPoint/Canva for diagrams; spreadsheet/Word/Docs for contract tables; Postman/Bruno/Insomnia may be used later for testing.
Scope Rule	Plan only endpoints required by the approved semester-project scope. Do not invent unused features or duplicate the IT 108 database design.

4. Pre-Laboratory Requirements

- Approved system title, problem statement, scope, functional/non-functional requirements, and user roles from Activity 5.
- System-specific Use Case Diagram from Activity 5 and Flutter UI-flow/wireframes from Activity 6.
- Architecture blueprint and prescribed Flutter Web → REST API → PostgreSQL stack from the earlier architecture/technology-stack activities.
- Instructor-approved backend framework/runtime for the REST API.
- The current PostgreSQL table/entity names or data domains from IT 108 for reference only; do not recreate the ERD in this activity.
- At least one critical user workflow that can be traced from the Flutter Web interface to a database operation and back.

5. Part A – Server-Side Responsibility Map

Before drawing the API flow, assign each concern to a clear server-side responsibility. Use the actual technologies and project terms your team plans to implement.

Server-Side Concern	Your Planned Component / Decision	Notes / Evidence
Request entry / routing	FastAPI Router	Example: POST /api/reports; route registration
Controller / handler	FastAPI Route Handler	Parses request; calls service; returns HTTP response
Authentication	JWT Authentication	Token/session verification; identity extraction
Authorization	Role-Based Access Control	Role/permission checks for protected operations
Server-side validation	FastAPI Validation	Required fields, formats, allowed values, cross-field rules
Service / business logic	FastAPI Service Layer	Approval, recommendation, calculations, workflow rules
Data access	Repository / Query Layer	Repository/query layer; parameterized query or ORM access
PostgreSQL data domain	Existing IT 108 domains	Reference table/entity names from IT 108 only
External service, if any	Machine Learning Service	Maps/GIS, ML service, email/SMS, weather, storage, etc.
Response / error handling	FastAPI Response Handler	Status code, response JSON, safe error message, logging

6. Part B – Request-Lifecycle Tracing Exercise

Select one critical workflow from your own system. Examples: log in, submit a report, create a record, approve a request, retrieve a map/list, update a profile, or request a recommendation.

Selected workflow: Household Vulnerability Assessment and Classification

Flutter screen / user action: Assessment Form → Submit Assessment

Expected final result shown to the user: The system displays the household's vulnerability classification as Low, Medium, or High, and the result can be used for aid prioritization/distribution.

Step	Actor / Component	Action or Data	Method / Route / Mechanism	Validation / Security	Expected Result
1	LGU Staff	Opens Household Details	Flutter Web	JWT session checked	Household information displayed
2	LGU Staff	Opens Assessment Form	Flutter Web	User must be authorized LGU Staff	Assessment form displayed
3	Flutter Client	Enters vulnerability indicators	Client-side form validation	Required fields checked	Form is ready for submission
4	Flutter Client	Sends assessment data	POST /api/households/{id}/assessments over HTTPS/JSON	Bearer JWT token included	Request reaches FastAPI
5	FastAPI Router / Handler	Receives request	REST endpoint	Token authentication and role authorization	Request accepted or rejected
6	FastAPI	Validates assessment	server-side validation	Required fields, valid values, household ID	Valid assessment proceeds
7	Assessment Service	Processes assessment	Service/business logic	Checks household and assessment rules	Assessment prepared for classification
8	ML Service	Classifies household	HTTPS/JSON	Valid assessment data	Returns Low/Medium/High
9	PostgreSQL	Saves assessment and classification	SQL / Repository	Protected database access	Data stored
10	FastAPI → Flutter	Returns result	200 OK + JSON	Safe response	Flutter displays Vulnerability Result

Tracing Guidance

A strong trace normally includes: (1) user action in Flutter, (2) client-side validation, (3) request creation with method/path/headers/body, (4) route matching, (5) authentication/authorization, (6) server-side validation, (7) service/business-rule execution, (8) PostgreSQL query or transaction, (9) response generation with status + JSON, and (10) Flutter state update and user feedback.

7. Request–Response Trace Check

☑The trace uses one real use case from Activity 5.

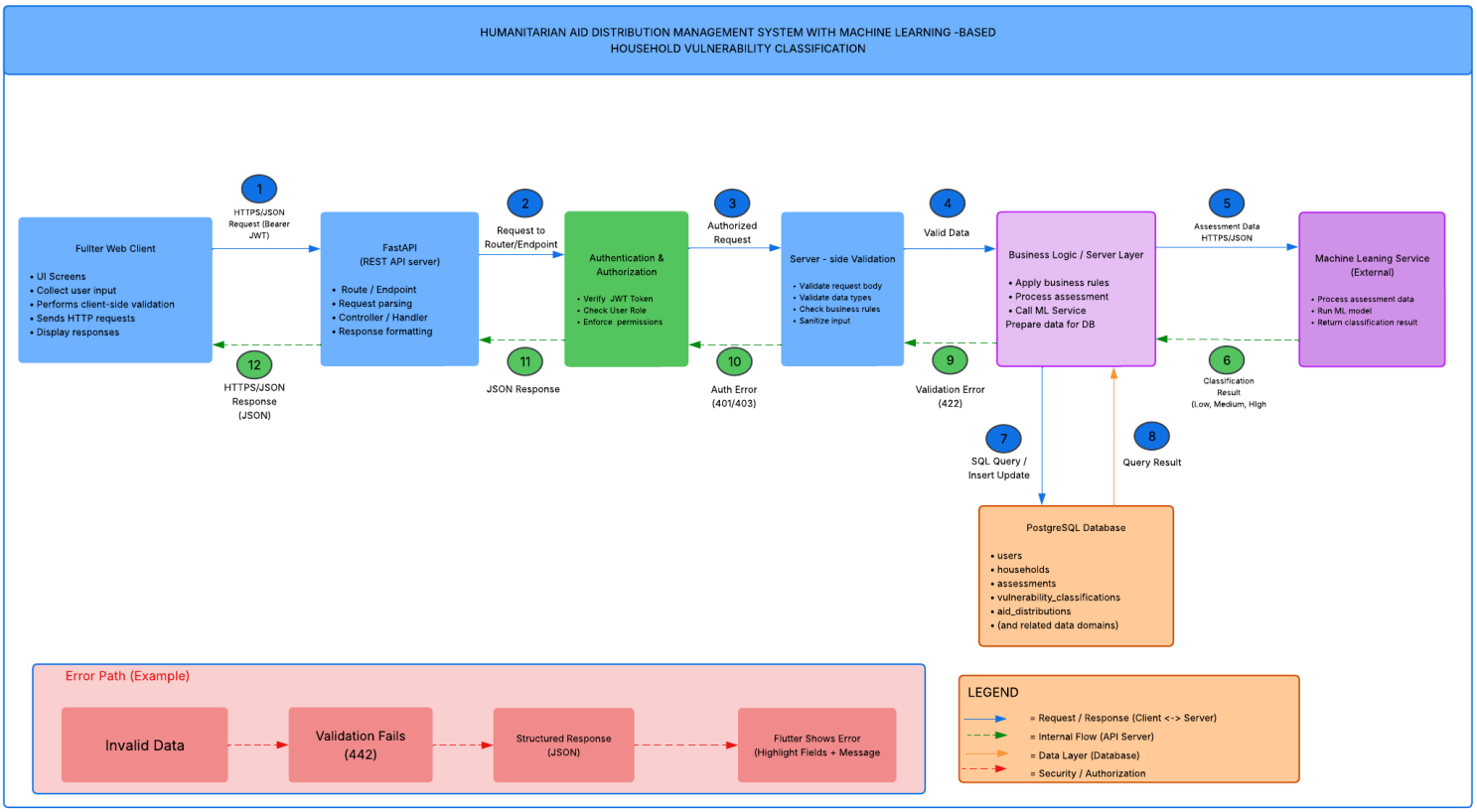
☑The starting screen exists in the Activity 6 UI flow.

☑ The endpoint has a meaningful HTTP method and resource-oriented path.
☑ Server-side validation is shown even when Flutter also validates the form.
☑ The database step references existing IT 108 data domains rather than recreating the schema.
☑ Both success and at least one failure path are identified.

8. Part C – Construct the API Flow Diagram

Create one clean, readable API Flow Diagram showing how a real request travels from Flutter Web through the REST API backend to PostgreSQL, and how the response returns to the interface. The diagram must describe your own project—not a generic sample.

No.	Required Element	What Must Be Shown
1	Flutter Web client	Identify the actual screen/widget or user action that starts the request.
2	Request	Show HTTP method, route/path, HTTPS, headers/token when required, and JSON/body or query parameters.
3	Route / endpoint	Show where the request enters the server-side application.
4	Controller / handler	Show request parsing, response coordination, and delegation to the service layer.
5	Validation & security	Show server-side validation, authentication, authorization, and protected operations where applicable.
6	Service / business logic	Show the rule, calculation, approval, recommendation, or workflow operation performed.
7	Data access	Show repository/query/data-access responsibility without embedding raw database design in the diagram.
8	PostgreSQL	Reference the relevant IT 108 data domain/table/entity names.
9	External service	Include only if the selected workflow actually uses one.
10	Response	Show returned status code, JSON/result, and Flutter state/UI feedback.
11	Error path	Show at least one structured failure path such as invalid input, unauthorized access, not found, conflict, or server error.
12	Legend	Use consistent shapes, arrows, and labels.


HUMANITARIAN AID DISTRIBUTION MANAGEMENT SYSTEM WITH MACHINE LEARNING -BASED HOUSEHOLD VULNERABILITY CLASSIFICATION
Fullter Web Client
• UI Screens
• Collect user input
• Performs client-side validation
• Sends HTTP requests
• Display responses
1
HTTPS/JSON Request (Bearer JWT)
FastAPI (REST API server)
• Route / Endpoint
• Request parsing
• Controller / Handler
• Response formatting
2
Request to Router/Endpoint
Authentication & Authorization
• Verify JWT Token
• Check User Role
• Enforce permissions
3
Authorized Request
Server - side Validation
• Validate request body
• Validate data types
• Check business rules
• Sanitize input
4
Valid Data
Business Logic / Server Layer
• Apply business rules
• Process assessment
• Call ML Service
Prepare data for DB
5
Assessment Data HTTPS/JSON
Machine Leaning Service (External)
• Process assessment data
• Run ML model
• Return classification result
6
Classification Result (Low, Medium, High)
7
SQL Query / Insert Update
8
Query Result
9
Validation Error (422)
10
Auth Error (401/403)
11
JSON Response
12
HTTPS/JSON Response (JSON)
PostgreSQL Database
• users
• households
• assessments
• vulnerability_classifications
• aid_distributions
• (and related data domains)
Error Path (Example)
Invalid Data
Validation Fails (442)
Structured Response (JSON)
Flutter Shows Error (Highlight Fields + Message
LEGEND
= Request / Response (Client <-> Server)
= Internal Flow (API Server)
= Data Layer (Database)
= Security / Authorization

9. Reference Example – Do Not Copy as Your Final Output

The figure below demonstrates the expected level of clarity for a Flutter Web → REST API → PostgreSQL request lifecycle. Replace all labels with the actual endpoint, business logic, security checks, data domains, and response behavior of your own system.

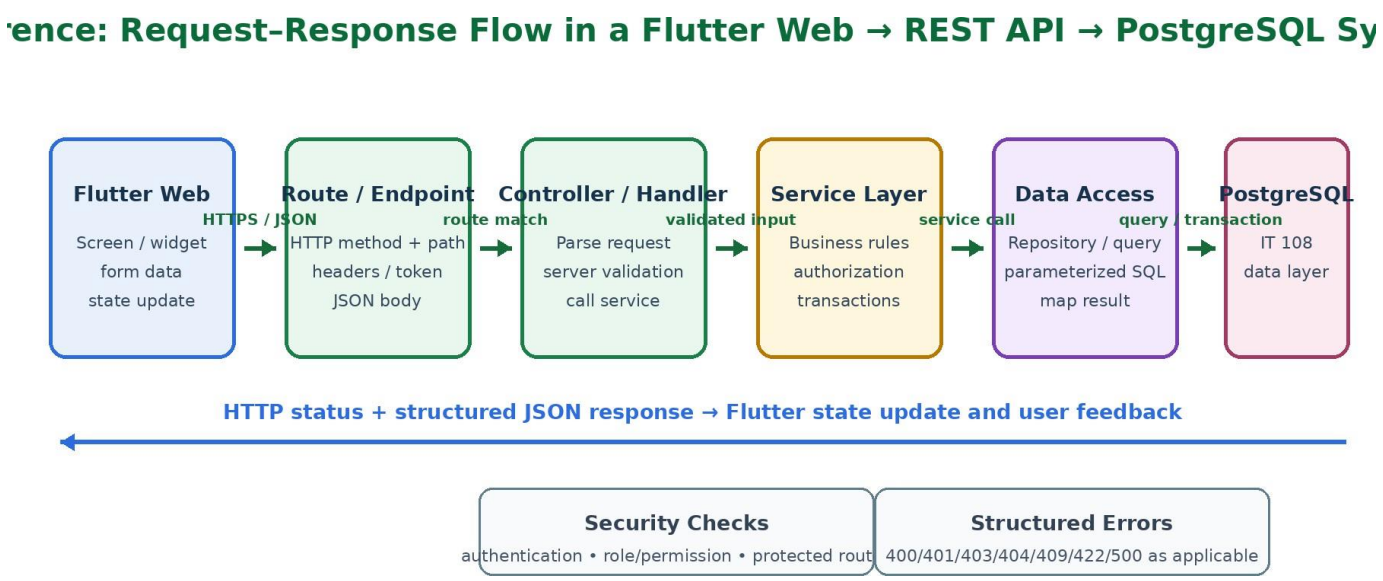


Figure 1. Conceptual request–response path for a Flutter Web client using a REST API and PostgreSQL.

Minimum Flow Rule

Your final diagram must show at least one complete path: Flutter screen/user action → HTTP request → endpoint/route → handler/controller → server validation/security → service/business logic → PostgreSQL or external service → HTTP status + JSON response → Flutter state/UI result. Every arrow on this path must have a meaningful label.

10. Recommended API Flow Labels

Area	Possible Label	What It Represents
Client	Flutter Web screen / widget	User input, event, client-side validation, state update
Transport	HTTPS + REST + JSON	Method, path, headers, body/query parameters
Routing	Endpoint / route	Matches request to server-side handler
Handler	Controller / request handler	Parses request and coordinates response
Service	Service / use-case layer	Business rules, permissions, workflow logic
Data Access	Repository / query layer	Reads/writes PostgreSQL through safe parameterized access or ORM
Security	Auth middleware / guards	Identity, token verification, role/permission checks
Response	HTTP status + JSON	Success result or structured error for Flutter

11. Part D – Semester-Project Endpoint Inventory

List the initial endpoints needed to support the approved core features. Use resource-oriented paths and select methods based on the intended action. Do not create endpoints for features outside the approved scope.

ID	Requirement / Use Case	HTTP Method	Route / Path	Authorized Role(s)	Handler / Service	PostgreSQL Data Domain
EP-01	User Login	POST	/api/auth/login	All users	Auth Handler / Auth Service	users
EP-02	View Household List	GET	/api/households	LGU Admin, LGU Staff	Household Handler / Household Service	households
EP-03	Register Household	POST	/api/households	LGU Staff	Household Handler / Household Service	households
EP-04	View Household Details	GET	/api/households/{id}	LGU Admin, LGU Staff, authorized Household	Household Handler / Household Service	households
EP-05	Submit Vulnerability Assessment	POST	/api/households/{id}/assessments	LGU Staff	Vulnerability Handler / Classification Service	assessments
EP-06	Get Vulnerability Result	GET	/api/households/{id}/vulnerability	LGU Admin, LGU Staff, authorized Household	Vulnerability Handler / Classification Service	vulnerability_classifications
EP-07	Record Aid Distribution	POST	/api/households/{id}/aid-distributions	LGU Staff	Aid Handler / Distribution Service	aid_distributions
EP-08	View Reports	GET	/api/reports	LGU Administrator	Report Handler / Report Service	Related project data domains

12. REST Method and Route Selection Guide

Method	Typical Purpose	Example	Design Note
GET	Retrieve a resource or collection	GET /api/reports; GET /api/reports/{id}	Should not create side effects.
POST	Create a resource or trigger a non-idempotent operation	POST /api/reports	Usually returns 201 Created when a resource is created.
PUT	Replace a complete resource when the API is designed that way	PUT /api/users/{id}	Use consistently; often idempotent.
PATCH	Partially update a resource	PATCH /api/reports/{id}	Use for partial changes such as status or selected fields.
DELETE	Remove a resource when deletion is allowed	DELETE /api/reports/{id}	Often returns 204 No Content after successful deletion.

Route Naming Reminder

ID	Requirement / Use Case	HTTP Method	Route / Path	Authorized Role(s)	Handler / Service	PostgreSQL Data Domain

Prefer nouns/resources in paths, keep naming consistent, avoid exposing raw database table operations directly, and make endpoint names understandable from the system's domain. Exact conventions may vary with the selected backend framework, but the contract must remain clear and consistent.

13. Part E – Initial API Contract

Document at least FOUR core endpoints from the inventory. Two of the four must be written in full detail using the contract template below; the other two may be summarized in the endpoint inventory if time is limited during the laboratory period.

Contract ID / Feature	Record Aid Distribution
HTTP Method + Route	Register Household
Purpose / Related Use Case	Allows authorized LGU Staff to record aid given to a household.
Authorized Role(s)	LGU Staff
Required Headers	Example: Content-Type: application/json; Authorization: Bearer <token> when protected
Path / Query Parameters	Name, type, required/optional, meaning
Request JSON Body	Fields, types, required/optional, example values—not real personal data
Server-Side Validation	Required fields, allowed values, relationships, business-rule checks
Success Response	Status code + response JSON fields / result
Error Responses	Possible status codes + structured error code/message/details
PostgreSQL Data Domain	Reference relevant IT 108 entity/table/domain names only

14. Contract Quality Rules

- The contract must be understandable without looking at the implementation code.
- The route and HTTP method must agree with the feature and request-lifecycle trace.
- Client-side validation from Activity 6 must be repeated or strengthened on the server when the rule protects data integrity or access.
- Do not return passwords, tokens, secrets, internal stack traces, raw SQL, or unnecessary private fields in responses.
- Use consistent field names and JSON types across requests and responses.
- Document expected failure cases instead of treating every failure as a generic 500 error.

15. HTTP Status Codes and Structured Responses

Status	Typical Meaning in the API Contract
200 OK	Successful retrieval/update when a response body is returned.
201 Created	New resource successfully created.
204 No Content	Successful operation with no response body, commonly deletion or update.
400 Bad Request	Malformed request or invalid request syntax/structure.
401 Unauthorized	Authentication is missing or invalid.
403 Forbidden	User is authenticated but lacks permission.
404 Not Found	Requested resource does not exist or is not available to the requester.
409 Conflict	Request conflicts with current resource state, such as duplicate or invalid transition.
422 Unprocessable Content	Request is structurally valid but fails semantic/field validation when this convention is used.
500 Internal Server Error	Unexpected server failure; return a safe message and log details server-side.

16. JSON Request / Response and Error Design

Use valid JSON with consistent field names and types. Your actual contract may use a different structure, but success and error responses must be predictable for the Flutter client.

Example Success	Example Structured Error

HTTP 201
{
"id": 125,
"status": "submitted",
"message": "Report created"
}
	HTTP 422
	{
	"error": {
	"code": "VALIDATION_ERROR",
	"message": "Some fields are invalid",
	"details": { "location": "Required" }
	}
	}

Status-Code Reminder

The table provides common REST conventions, not a rule that every project must use every status. Select the codes that accurately describe your API behavior, document them in the contract, and apply them consistently.

17. Part F – Server-Side Validation and Error Matrix

For one form or write operation from Activity 6, identify the rules that must be enforced on the server even if Flutter already performs client-side validation.

Field / Rule	Client-Side Check	Server-Side Check	Business / Security Reason	Error Status	Safe Error Message / Code
Username	Required	Required and valid credentials	Prevent unauthorized login	401	INVALID_CREDENTIALS
Password	Required	Verify password	Protect account access	401	Invalid username or password.
JWT Token	Token exists	Verify token and expiration	Protect API resources	401	UNAUTHORIZED
User Role	UI restricts options	Server checks role	Prevent unauthorized operations	403	FORBIDDEN
Head of Family	Required	Required	Complete household record	422	HEAD_REQUIRED
Household Size	Positive number	Must be greater than 0	Prevent invalid household data	422	INVALID_HOUSEHOLD_SIZE
Address	Required	Required	Complete household information	422	ADDRESS_REQUIRED
Contact Number	Valid format	Validate format	Maintain valid contact information	422	INVALID_CONTACT
Assessment Indicators	Required	Required and allowed values	Ensure ML receives valid data	422	VALIDATION_ERROR
Household ID	Selected	Must exist	Prevent assessment for nonexistent household	404	HOUSEHOLD_NOT_FOUND
Aid Item	Required	Required	Complete distribution record	422	AID_ITEM_REQUIRED
Aid Quantity	Greater than 0	Must be greater than 0	Prevent invalid distribution quantity	422	INVALID_QUANTITY

Distribution Date	Valid date	Valid date	Maintain valid distribution history	422	INVALID_DATE

18. Part G – Conceptual API-to-PostgreSQL Mapping

Connect API operations to the existing PostgreSQL data layer from IT 108 at a conceptual level. Do not recreate the ERD, normalization, keys, constraints, or physical schema here.

Endpoint	Service / Business Operation	Data Action	IT 108 Data Domain / Entity	Returned Result
POST /api/auth/login	Authenticate User	Read	users	JWT + user role
GET /api/households	Retrieve Households	Read	households	Household list
POST /api/households	Register Household	Insert	households	Created household
GET /api/households/{id}	View Household	Read	households	Household details
POST /api/households/{id}/assessments	Process Assessment	Insert	assessments	Assessment record
POST /api/households/{id}/assessments	Save Classification	Insert	vulnerability_classifications	Low/Medium/High result
GET /api/households/{id}/vulnerability	Retrieve Classification	Read	vulnerability_classifications	Vulnerability result
POST /api/households/{id}/aid-distributions	Record Aid	Insert	aid_distributions	Distribution record
GET /api/reports	Generate Reports	Read	Related project data domains	Report data

Endpoint	Service / Business Operation	Data Action	IT 108 Data Domain / Entity	Returned Result

IT 108 Boundary

Reference the approved IT 108 table/entity or data-domain names only to show where the API reads or writes. Do not recreate or redesign the database schema in this Activity 7 manual.

19. Laboratory Procedure

Phase 1 – Select a critical workflow (10 minutes)
Choose one high-value use case that begins in an Activity 6 Flutter screen and requires the REST API and PostgreSQL.

Phase 2 – Trace the request lifecycle (20 minutes)
Complete Part B from user action to Flutter request, route, validation/security, service logic, data access, response, and UI feedback.

Phase 3 – Draft the API Flow Diagram (25 minutes)
Draw the complete system-specific request/response path. Label methods, paths, data, validation, security checks, results, and errors.

Phase 4 – Plan endpoints (20 minutes)
Build the endpoint inventory for the approved core features. Review method selection, path naming, roles, services, and data domains.

Phase 5 – Write the initial API contract (30 minutes)
Document at least four core endpoints; fully detail two using the contract template.

Phase 6 – Validate responses and errors (10 minutes)
Check JSON bodies, HTTP status codes, headers, field validation, authorization, and structured errors.

Phase 7 – Review API-to-PostgreSQL mapping (10 minutes)
Confirm that the API references the current IT 108 data domains without duplicating database design.

Phase 8 – Final review and submission preparation (15 minutes)
Use the checklist, update the diagram and contract, export required files, and commit documentation to GitHub.

20. Final Quality Checklist

☑ The API Flow Diagram uses the official system title and a real critical workflow.	☑ JSON request/response fields and data types are consistent.
☑ The starting Flutter screen/user action exists in Activity 6.	☑ Success and failure status codes are documented.
☑ At least one complete request-and-response path is visible and labeled.	☑ Errors are structured and do not expose internal or sensitive details.
☑ HTTP methods and route paths are meaningful and consistent.	☑ PostgreSQL references agree with the IT 108 data layer and do not duplicate schema design.
☑ Controllers/handlers and service/business logic are not treated as the same responsibility without explanation.	☑ Endpoint inventory is traceable to approved requirements/use cases.
Server-side validation is included even when Flutter validates the same input.	☑ At least four core endpoints are documented and two are fully detailed.
☑ Authentication and authorization are shown for protected endpoints.	☑ Every group member can explain the selected request lifecycle and API contract.

21. Submission Requirements

- PDF or PNG of the final API Flow Diagram in readable landscape orientation.
- Editable diagram source file (.drawio, .pptx, .fig, or native file of the chosen tool).
- Completed request-lifecycle tracing worksheet.
- Endpoint inventory and Initial API Contract with at least four core endpoints; at least two endpoints must contain full request/response, validation, status-code, and error details.
- Completed server-side validation/error matrix and conceptual API-to-PostgreSQL mapping.
- Commit the files to the project GitHub repository under a folder such as docs/api/ or docs/backend/.
- Submit the repository link and required files through the class submission platform using the naming convention: IT110_LA7_Group##_SystemName.

Academic Integrity and Competency Validation

The API design must represent the team's own approved system and must be consistent with its requirements, use case diagram, Flutter UI flow, and IT 108 data layer. AI tools may be used to clarify concepts or improve documentation, but every endpoint, validation rule, role restriction, request/response field, and flow decision must be reviewed and understood by the team. The instructor may ask any member to trace a request or explain an endpoint without assistance.

22. Scoring Rubric – 80 Points

Criterion	Evidence for Full Credit	Points
API flow correctness and completeness	The diagram accurately traces Flutter → REST API → PostgreSQL → response, including routes, handlers, validation/security, service logic, data access, and error/success paths.	15
Endpoint planning and REST design	Core endpoints are traceable to approved features; methods and resource-oriented paths are appropriate, consistent, and limited to the project scope.	12
Initial API contract completeness	At least four core endpoints are documented and the required detailed contracts clearly define purpose, roles, parameters, headers, request/response bodies, validation, status codes, and errors.	15
HTTP semantics and JSON design	Methods, status codes, request/response fields, JSON types, headers, and result structures are technically coherent and consistently documented.	10
Server-side validation, security, and errors	Validation, authentication, authorization, protected operations, safe error messages, and structured failure responses are clearly planned.	10
Conceptual API-to-PostgreSQL mapping	Endpoints and service operations map correctly to existing IT 108 data domains without duplicating database design.	8
Traceability and implementation readiness	The API plan aligns with Activity 5 requirements/use cases and Activity 6 Flutter screens and can guide subsequent backend implementation.	5
Documentation quality and technical clarity	Tables, labels, terminology, diagrams, and file organization are readable, consistent, and professionally prepared.	5

23. Post-Laboratory Reflection

Which endpoint in your system is the most security-sensitive, and what authentication/authorization checks must occur before its business logic runs? - The most security-sensitive endpoint is the user login endpoint because it controls access to the system. The API must validate the username and password, verify the user's account, and issue a secure authentication token. For protected endpoints, the server must verify the JWT token and check the user's role before allowing the business logic to execute.

Which validation rule must be enforced on the server even if Flutter already checks it, and why? - The household assessment validation rules must be enforced on the server even if Flutter checks them. This prevents invalid or incomplete assessment data from reaching the ML service and PostgreSQL. Server-side validation also prevents users from bypassing Flutter validation by directly sending API requests.

Which API response will be the most difficult for the Flutter client to handle correctly, and what UI state should be shown? - The ML service/API failure response may be the most difficult because the vulnerability classification cannot be completed if the ML service is unavailable. Flutter should show an API/Server Error state with a clear message and a Retry action while preserving the assessment information when possible.

Where is the clearest boundary between the controller/handler and service/business-logic responsibilities in your selected workflow? - The controller/handler should receive the HTTP request, parse the JSON data, call validation and the appropriate service, and return the HTTP response. The service layer should handle the actual assessment workflow, business rules, ML service request, and saving the result. This keeps HTTP handling separate from business logic.

What API contract detail must your team finalize before beginning the next server-side implementation activity?

- The team must finalize the endpoint paths, HTTP methods, JSON request/response fields, authentication requirements, roles, validation rules, and error status codes**.**